

TDA ABSTRACCIÓN

ABSTRACCIÓN → LA TV, EL PERRO

ABSTRACCIÓN DE DATOS → OCULTAR CARACTERÍSTICAS DEL PERRO (ABSTRACCIÓN) PARA SER INTELIGENTES

COMO ESTÁN ESTRUCTURADA LA TV POR EJEMPLO

ENCAPSULAMIENTO → Hay una parte que está oculta

TDA → INTERFAZ → h (datos y funciones); APLICACIÓN DE DATOS O
→ IMPLEMENTACIÓN $(.c)$

FC = FUNCIÓN

VENTAJAS = SE PUEDE REUTILIZAR, SE PUEDEN MANTENER MÁS FACIL LOS DATOS (SE MANTIENE ALGO SIN REESCRIBIR EL RESTO)

TDA → TIPO DE DATO → CARACT DEL TIPO DE OBJETO (RADIO)
→ ATRIBUTOS (VOLUMEN, FRECUENCIA)

→ APLICACIÓN → TRABAJO CON LOS DATOS DE LOS ATRIBUTOS (0-100 EL VOLUMEN)

→ OPERACIONES → VOLÚMENES PERMITIDAS SOBRE NUESTRO OBJETO (AUMENTAR, SUBIR VOLUMEN)

→ PRE Y POS CONDICIONES → LO QUE SE HACE ANTES Y DESPUÉS DE LA CONDICIÓN (PRIMERO PRENDER LA FOLLA Y LUEGO SUBIR VOLUMEN)

FD: INTERFAZ (PÚBLICO)
→ FRACCIÓN a/b

D = DENOMINADOR N = NUMERADOR

STRUCT FRACCIÓN STRUCT;
TYPEDF FRACCIÓN STRUCT * FRACCIÓN;

FRACCIÓN CREAR (INT D, INT N);
INT DEF (FRACCIÓN F);
INT NUMERO (FRACCIÓN F);
VOID IMPRIMIR (FRACCIÓN F);

IMPLEMENTACIÓN (OCULTO)
→ FRACCIÓN $.c$

#INCLUDE "FRACCIÓN $.h$ "



```

MAIN.C
#include "Fraction.h"
int A;
A = defn;

```

STRUCT. RESTRUCTING AUTO \$
INT. MALL;
CHAR. MODEL (20);
FLAT. ANSLO;
3;

```

AUTO.C
#include <stdio.h>
#include <stdlib.h>
#include "ntg.h"

```

$\{P\}$
Int w ;
Int x $\leftarrow$ random;

TRUJILLO (SECRETARÍA DE EDUCACIÓN PÚBLICA)
CALLE 100, / 100-100-100
DE LA EDUCACIÓN

main.c
#include <stdio.h>
" " <stdio.h>
" " "stdio.h"
main() {

RETURNED
3

CLIMATE H

STRUCT. RESTORATION PROGRAM

11. Functionalised Polymers

VOID Molar Protona (Protona P):

INT. CALCULAR VARIACIONES (PARSONA P):

// GRT y SRT

VOID SET_NUMBER (PERSONA P, char n[20])

VOID SET-ANTIUNION (PERSON P, INT V);

VED SAT - APRIL 20 (" " " " [20])

INT. GRT. AUTHORITY (PROSECUT.)

```
void SRT_Eval ( " ", INT e).
```

VOID SET - PASO (" " , FLOAT P)

Veid SRT - DNI (" " , Long d)

CHAR + GFT - NOMINACE (PFRSUA P)

CAR * GET - APELL DO (" ")

```
INT GET_EDAD ( " " );
```

```
GET_PRSO(, " ");
```

```
float GET_DNI (" ");
```

//CLISTFE.C

INCLUDE "CLIENT.H"

ESTRUTURA

STRUCTURE PISTONNA

CHGR NUMBER [20]

CHAR APPELLIDO [20]

Tu7 EDAD:

Floor Plate:

one DNF: I.T. ANTIGUERRA

2

15. 11. 1950

Proc. Cogn. Process (CHAR, Numb, 20), CHAR, APPL, 20, INT, READ, PLAN, PLAN, LATER, 20, 20

main - main = malloc(sizeof(struct FSTRUCTURA PERSONA));

$\text{Antiferromagnet} = \text{Antiferromagnetic}$
 $\text{Ferromagnet} = \text{Ferromagnetic}$ $\rightarrow$ Antiferromagnet = Antiferromagnetic

total $\rightarrow$ ROAD = ROAD;
* $\rightarrow$ POK = POK;

$$\text{then } \rightarrow P_{ASO} = P_{ASO};$$

teris $\rightarrow DII = DNI$;

STREPT (materis) → NOMBRE, NOMBRE

STROM (materie → APRILIO, APRILIO);

STICKY

Actual time:

2

3


```

// PROBABILIDAD
void MostrarPersona (Persona P) {
    PRINTF ("%s... %s... %d... %d... %d", P->NOMBRE, P->APELLIDO, P->EDAD, P->PRSO, P->DNI);
}

// GET, SET
void SET_EDAD (Persona P, int e) {
    P->EDAD = e;
}

int GET_EDAD (Persona P) {
    return P->EDAD;
}

void SET_DNI (Persona P, long d) {
    P->DNI = d;
}

long GET_DNI (Persona P) {
    return P->DNI;
}

void SET_NOMBRE (Persona P, char STR[]) {
    strcpy (P->NOMBRE, STR);
}

char* GET_NOMBRE (Persona P) {
    return P->NOMBRE;
}

```

```

// MAIN.C
#include "CLIFTR.H"

int main() {
    Persona persona1 = CreaPersona ("Nicolas", "Perez", 28, 0177, 3545999);
    MostrarPersona (persona1);
    PRINTF ("MUESTRA LA EDAD CON EL GET");
    PRINTF ("%d ES LA EDAD", GET_EDAD (persona1));

    PRINTF ("MODIFICO LA EDAD CON EL SET / LA MUESTRO");
    SET_EDAD (persona1, 90);
    PRINTF ("%d ES LA EDAD", GET_EDAD (persona1));

    PRINTF ("%d ES SU DNI", GET_DNI (persona1));
    SET_DNI (persona1, 4238758);
    PRINTF ("%d ES SU DNI", GET_DNI (persona1));

    PRINTF ("%s ES SU NOMBRE", GET_NOMBRE (persona1));
    SET_NOMBRE (persona1, "Luciano");
    PRINTF ("%s ES SU NOMBRE", GET_NOMBRE (persona1));

    PRINTF ("CANT VACAS : ", CALCULAR_VACACIONES (persona1));
}

```

// FUNCION DNE VACACIONES

```

int CALCULAR_VACACIONES (Persona P) {
    IF (P.ANTIGUEDAD < 5)
        RETURN 14;

    IF (P.ANTIGUEDAD > 5 && P.ANTIGUEDAD < 10)
        RETURN 21;

    IF (P.ANTIGUEDAD > 10 && P.ANTIGUEDAD < 20)
        RETURN 28;

    IF (P.ANTIGUEDAD > 20)
        RETURN 35;
}

```

TDA COMPLETO AUTO

//AUTO.H

```
//SE DECLARAN FUNCIONES Y PROCEDIMIENTOS
//DECLARACION DE LA ESTRUCTURA AUTO
STRUCT AUTOFESTRUCTURA;
//DEFINIMOS UN TIPO DE DATO QUE ES UN PUNTERO (AUTO) A PUNTA A LA ESTRUCTURA (SU CUPADO ESTARA EN EL .C)
TYPEDEF STRUCT AUTOFESTRUCTURA *AUTO;
//CREAMOS UN AUTO Y PROPARAMOS COMO ARGUMENTOS LO QUE SE NECESITA PARA CREAR UN AUTO.
Auto CREAR_AUTO (INT MODELO, CHAR NOMBRE[20], INT CANT CHAQUROS);
//MOSTRAMOS EL AUTO Y OBTENEMOS UN AUTO COMO ARGUMENTO Y QUE EN UN PUNTERO.
VOID MOSTRAR_AUTO (AUTO a);
//CREAMOS AUTO POR TECLADO
Auto CREAR_AUTO_POR_TECLEADO();
```

//AUTO.C

```
#INCLUDE "AUTO.H"
//INCLUIREMOS SOLO EL .H (YA QUE ESTRUCTURAS, FUNCIONES Y PROCEDIMIENTOS)
//LUEGO DE LA ESTRUCTURA
STRUCT AUTOFESTRUCTURA {
    INT MODELO;
    CHAR NOMBRE[20];
    INT CANT CHAQUROS;
};
```

```
//FUNCIÓN CREAR AUTO
Auto CREAR_AUTO (INT MODELO, CHAR NOMBRE[20], INT CANT CHAQUROS) {
    //RESERVAMOS MEMORIA Auto auto = malloc (sizeof (STRUCT AUTOFESTRUCTURA));
    //LE DAMOS VALOR QUE RECIBIMOS COMO ARGUMENTOS.
    auto->MODELO = MODELO;
    strcpy (auto->NOMBRE, NOMBRE);
    auto->CANT CHAQUROS = CANT CHAQUROS;
    RETURN auto; //RETORNAMOS EL PUNTERO auto.
}
```

```
//MOSTRAMOS UN AUTO
VOID MOSTRAR_AUTO (AUTO a) {
    PRINTF ("%d ... %s ... %d", a->MODELO, a->NOMBRE, a->CANT CHAQUROS);
}
```

```
//CREAMOS AUTO POR TECLADO
Auto CREAR_AUTO_POR_TECLEADO () {
    INT MODELO = 0, CANT NOMBRE[20] = "", INT CANT CHAQUROS = 0;
```

```
    PRINTF ("INGRESA MODELO: ");
    scanf ("%d", &MODELO);
    PRINTF ("INGRESA NOMBRE: ");
    gets (NOMBRE);
    PRINTF ("INGRESA CANTIDAD DE CHAQUROS: ");
    scanf ("%d", &CANT CHAQUROS);
    Auto auto = malloc (sizeof (STRUCT AUTOFESTRUCTURA));
    RETURN CREAR_AUTO (MODELO, NOMBRE, CANT CHAQUROS);
}
```


CONCESIONARIA

// CONCESIONARIA.H

STRUCT CONSTRUCCION;

TYPEDF struct CONSTRUCCION * CONGE;

CONGE CONGE (CAR d [20], INT u, CHAR m [20]);

VOID MOSTRAR CONGE (CONGE C);

// CONCESIONARIA.C

#INCLUDE "Auto.h"

#INCLUDE "CONCESIONARIA.H"

STRUCT CONSTRUCCION C;

CHAR DIR d [20];

INT AUTUA;

CHAR NOMBRE [20];

AUTO auto1 [20]; // HASTA 20 AUTOS PARA TENER

INT CANTIDAD DE AUTOS;

{
CONGE CONGE (CAR d [20], INT u, CHAR m [20]);
CONGE CONGE = malloc (sizeof struct CONSTRUCCION);

strcpy (CONGE -> DIR, d);

strcpy (CONGE -> NOMBRE, m);

CONGE -> AUTUA = u;

CONGE -> CANTIDAD DE AUTOS = 0;

// MODO PARA AUMENTAR LA CANT DE AUTOS

CHAR OP CIN = 'S';

WHILE (OP CIN != 'S') {

PRINTF ("DESEA CONGE ALGUN AUTO? S/N: ");

scanf ("%c", &OP CIN);

IF (OP CIN != 'S') {

CONGE -> auto1 [CONGE -> CANTIDAD DE AUTOS] = CONGE -> auto1 [CONGE -> CANTIDAD DE AUTOS - 1];

CONGE -> CANTIDAD DE AUTOS = CONGE -> CANTIDAD DE AUTOS + 1;

}

}

RETURN CONGE;

}

VOID MOSTRAR CONGE (CONGE C)

PRINTF ("NOMBRE: %s, Direccion: %s, AUTUA: %d, NOMBRE: %s", C -> DIR, C -> NOMBRE, C -> AUTUA, C -> NOMBRE);

IF (C -> CANTIDAD DE AUTOS > 0)

PRINTF ("... AUTOS DE LA CONCESIONARIA: ");

FOR (INT I = 0; I < C -> CANTIDAD DE AUTOS; I++)

MOSTRAR CONGE (C -> auto1 [I]);

}

}

// MAIN.C

#INCLUDE "Auto.h"

#INCLUDE "CONCESIONARIA.H"

INT main () {

AUTO auto1 = CONGE -> auto1 (2000, "Ford", 3);

PRINTF ("MOSTRANDO EL AUTO");

MOSTRAR AUTO (auto1);

PRINTF ("CONGE AUTO PR-TRILADO");

AUTO auto2 = CONGE -> auto1 PR-TRILADO ();

PRINTF ("MOSTRANDO EL AUTO PR-TRILADO");

MOSTRAR AUTO (auto2);

PRINTF ("CONCESIONARIA CONGE");

CONGE C = CONGE (CAR "29 SEPTEMBER", 1335, "JIRA");

PRINTF ("MOSTRANDO CONCESIONARIA");

MOSTRAR CONGE (C);

RETURN 0;

}

6) TRIST DUCKWITZ FISTULICUTIVA;
7) TRIST STRICT DUCKWITZ FISTULICUTIVA * Duckwiti;
Duckwiti CROCODILUS (?)

Docente: C

```
void SKT_DNI_Destroy ( " ", INT DNI_n ) {  
    d -> DNI = DNI_n;  
}
```

3

MATERIA.H

```

STRUCT materiaEstrutura;
TIPEDRE struct " " * MATERIA;
MATERIA CRIA MATERIA (CHAR NOME[20], INT CODIGO);
VOID MOSTRA MATERIA (MATERIA m);
VOID ADELGAZAR DADOS ( " " );
CHAR * GET NOME ( " " );
INT GET CODIGO ( " " );
VOID SET NOME ( " " , CHAR NOME[20]);
VOID SET CODIGO ( " " , INT CODIGO);

```

MATERIA.C

```

#include "MATERIA.H"
#include "DESAFIO.H"
STRUCT materiaEstrutura {
    CHAR NOME[20];
    INT CODIGO;
    DESAFIO DESAFIO[5];
    INT CAPTADO;
};

MATERIA CRIA MATERIA (CHAR NOME[20], INT CODIGO) {
    MATERIA m = malloc(sizeof(STRUCT materiaEstrutura));
    m->CODIGO = CODIGO;
    strcpy(m->NOME, NOME);
    m->CAPTADO = 0;
    return m;
}

VOID MOSTRA MATERIA (MATERIA m) {
    printf("NOME: %s, CODIGO: %d", m->NOME, m->CODIGO);
    for (int i = 0; i < 5; i++) {
        printf("DESAFIO %d: ", i+1);
        MOSTRA DESAFIO (m->DESAFIO[i]);
    }
}

VOID ADELGAZAR DADOS (MATERIA m) {
    int opcao = 5;
    while (opcao != 0) {
        printf("OPCAO DESELETA 0-5: ");
        scanf("%d", &opcao);
        if (opcao == 1) {
            printf("Digite o nome: ");
            char nome[20];
            GET NOME (nome);
            strcpy(m->NOME, nome);
        }
        if (opcao == 2) {
            printf("Digite o codigo: ");
            int codigo;
            GET CODIGO (codigo);
            m->CODIGO = codigo;
        }
    }
}

CHAR * GET NOME (MATERIA m) {
    return m->NOME;
}

INT GET CODIGO (MATERIA m) {
    return m->CODIGO;
}

```

```

main.c
#include "MATERIA.H"
#include "DESAFIO.H"
int main() {
    MATERIA m = CRIA MATERIA("IPC", 101);
    MOSTRA MATERIA (m);

    SET NOME (m, "IPC DE WLA");
    SET CODIGO (m, 101);
    MOSTRA MATERIA (m);

    printf("Linha de MATERIA: %s, %d", m->NOME, m->CODIGO);

    ADELGAZAR DADOS (m);
    MOSTRA MATERIA (m);
    return 0;
}

```


DIAGRAMA DE TDA

Equipo
 Nombre Equipo: CHAR [20]
 Nombre Partido: CHAR [50]
 Cantidad Jugadores: INT
 Crear Equipo (char NombreEquipo [20], char NombrePartido [50]): EQUIPO
 Mostrar Equipo (EQUIPO e): VOID
 Asignar Jugadores (EQUIPO e): VOID

0
 :
 23
 (Posiciones)

JUGADOR

JUGADOR
 DNI: INT
 Nombre: CHAR [20]
 Apellido: CHAR [20]
 Crear Jugador (INT DNI, CHAR Nombre [20], CHAR Apellido [20]): JUGADOR
 Mostrar Jugador (JUGADOR): VOID

//JUGADOR.H

STRUCT JUGADOR ESTRUCTURA;
 TYPEDF STRUCT JUGADOR ESTRUCTURA *JUGADOR;
~~JUGADOR~~ CREAR_JUGADOR (LONG DNI, CHAR Nombre [20], CHAR Apellido [20]);
 VOID MOSTRAR_JUGADOR (JUGADOR);
 VOID SET_DNI (JUGADOR, LONG DNI);
 VOID SET_NOMBRE (" ", CHAR Nombre [20]);
 VOID SET_APELLIDO (" ", CHAR Apellido [20]);
 INT GET_DNI (" ");
 CHAR *GET_NOMBRE (" ");
 CHAR *GET_APELLIDO (" ");

//JUGADOR.C

#INCLUDE "JUGADOR.H"

STRUCT JUGADOR ESTRUCTURA;
 LONG DNI;
 CHAR Nombre [20];
 CHAR Apellido [20];

3

```

char *CrearJugador (long DNI, char Nombre[20], char Apellido[20]) {
    Jugador j = MakeStruct (struct Jugador Estructura);
    j.DNI = DNI;
    strcpy (j.Nombre, Nombre);
    " (j.Apellido, Apellido);
    return j;
}

```

```

void MostrarJugador (Jugador j) {
    printf ("%s\n", j.Nombre);
    " %s\n", j.Apellido);
}

```

```

void SetDNI (Jugador j, long DNI) {
    j.DNI = DNI;
}

```

```

void SetNombre (Jugador j, char Nombre[20]) {
    strcpy (j.Nombre, Nombre);
}

```

```

void SetApellido (Jugador j, char Apellido[20]) {
    strcpy (j.Apellido, Apellido);
}

```

```

long GetDNI (Jugador j) {
    return j.DNI;
}

```

```

char *GetNombre (Jugador j) {
    return j.Nombre;
}

```

```

char *GetApellido (Jugador j) {
    return j.Apellido;
}

```

//EQUIPO.H

```

struct Equipo Estructura;
typedef struct " " *Equipo;

```

```

Equipo CrearEquipo (char NombreEquipo[20], char NombreEstadio[20]);

```

```

void MostrarEquipo (Equipo e);

```

```

void AgregarJugadores (Equipo e);

```

```

void MostrarJugadores (Equipo e);

```

```

void SetNombreEquipo (Equipo e, char NombreEquipo[20]);

```

```

void SetEstadio (Equipo e, char Estadio[20]);

```

```

char *GetNombreEquipo (Equipo e);

```

```

char *GetEstadio (Equipo e);

```

//EQUIPO.C

```

#include "Equipo.H"

```

```

struct Equipo Estructura {
    char NombreEquipo[20];
    char Estadio[20];
    int CantidadJugadores;
    Jugador Jugadores[23];
}

```

```

Jugador CrearJugador = Paul Pichichi;
long DNI = 123456789, char NombreEquipo[20] = "Barca";
char Apellido[20] = "Pichichi";

```

```

printf ("Indicase DNI: ");

```

```

scanf ("%ld", &DNI);

```

```

printf ("Indicase Nombre: ");

```

```

scanf ("%s", NombreEquipo);

```

```

printf ("Indicase Apellido: ");

```

```

scanf ("%s", Apellido);

```

```

Jugador j = MakeStruct (struct Jugador Estructura);

```

```

return CrearJugador (DNI, NombreEquipo, Apellido);

```



```

Equipo CrearEquipo (char NombreEquipo [20], char NombreEstadio [50]) {
    Equipo e = malloc (sizeof (struct EquipoReservado));
    strcpy (e.NombreEquipo, NombreEquipo);
    e.NombreEstadio = NombreEstadio;
    return e;
}

```

```

void MostrarEquipo (Equipo e) {
    printf ("%s\n", e.NombreEquipo);
    for (int i = 0; i < e.CantidadJugadores; i++) {
        MostrarJugador (e.Jugadores [i]);
    }
}

```

```

void AgregarJugadores (Equipo e) {
    char opcion = 'S';
    while (opcion != 'N') {
        printf ("DESEA CARGAR UN JUGADOR? S->SI / N->NO\n");
        scanf ("%c", &opcion);
        if (opcion == 'S') {
            Jugador j = CrearJugador (30248959, "Nicolás", "GALLARDO");
            int posicion = e.CantidadJugadores;
            e.Jugadores [posicion] = j;
            e.CantidadJugadores = e.CantidadJugadores + 1;
        }
    }
}

```

```

void SetNombreEquipo (Equipo e, char NombreEquipoN [20]) {
    strcpy (e.NombreEquipo, NombreEquipoN);
}

```

```

void SetNombreEstadio ("", char NombreEstadioN [50]) {
    strcpy (e.NombreEstadio, NombreEstadioN);
}

```

```

char *GetNombreEquipo (Equipo e) {
    return e.NombreEquipo;
}

```

```

char *GetNombreEstadio ("") {
    return e.NombreEstadio;
}

```

```

void MostrarJugadores (Equipo e) {
    for (int i = 0; i < e.CantidadJugadores; i++) {
        printf ("Nombre: %s", e.Jugadores [i].Nombre);
        printf ("Apellido: %s", e.Jugadores [i].Apellido);
        printf ("DNI: %d", e.Jugadores [i].DNI);
    }
}

```

Int Min O3

```

Jugador N = Crea_Jugador_An_Inicial();
PRINTF("Tubos NURVO JUEGA:");

```

Master's Journals (Jugend N)

Adressor Invaders (jagador);
Master Invader ("");
RETURN;

LISTA

Diagram illustrating the flow of data from a source (Nodo 1) to a destination (Nodo 2) via a network (Nodo 3). The source (Nodo 1) contains data D_1 and a packet P_1 . The destination (Nodo 2) contains data D_2 and a packet P_2 . The network (Nodo 3) contains a packet P_2 and a packet P_1 . The flow is indicated by arrows: D_1 is sent to D_2 , and P_1 is sent to P_2 . The network (Nodo 3) is labeled "Nodo 3" and "Nodo 3".

LISTA E

Juguador j $\rightarrow$ Se cambia esto solo si es otro jugador
 No * Suficiente, $\frac{1}{N}$ interacción a cada uno nada

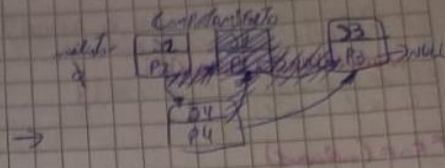
STREET LIGHTS

3 LISTA FINEC;

LISTA.H

[illegible]

MAIN: c
F² Tuckered "Jumbo" H
F² " " "Lista Enc. A"
Lista Enc. milita = Coroa Lista (1);
Sulamer Jigador = Coroa Jumbor ("Nico", 20);
Tuckered R. Principal (milita, Jigador 1);
Jumbor Jigador 2 = Coroa Sulamer ("Suebe", 33);
Tuckered R. Principal (milita, Jigador 2);
Jumbor Jigador 3 = Coroa Jumbor ("KPR", 40);
Tuckered R. Flak (milita, Jigador 3);
Sulamer Jigador 4 = Coroa Sulamer ("FLR", 15);
Tuckered (milita, Jigador 4, 1);
Guttes (milita, 20) // Bo de voguê. Para no Proximo
Mostrar Lista (milita).



ARCANOS

① FILE
 ② ARCHIVO → FOPEN
 ③ OPERACIONES → LECTURA
 ESCRITURA
 FGETC } CARACTERES
 FPUTC }
 LEITA = FGETC (en línea)
 FPUTC (LEITA, en línea)
 FGETS } CADENAS
 FPUTS }
 FGETS (FRASE, en línea)
 FPUTS (FRASE, en línea)
 FPRINTF } TEXTO con Formato
 scanf }
 FPRINTF (en línea, "%c", L)
 scanf ("%", "%c", &

FILE = TIPO STRUTTURA CON UN PUNTEGGIO
FILE* FOPEN (CARRA* APERTE, CARRA* MODO)
FILE* WOPEN (1);
wopen = FOPEN ("PROFESOR.TXT", "a")
R → LEGGERE (ARCHIVIO DA TROVARE) "r"
W → SCRITTURA "w"
A → SCRITTURA AL FINE
R+ → L/R (SOLO LEGGE ALL'ARCHIVIO)
W+ → L/R (PISA ALL'ARCHIVIO E SCRITTURA)
A+ → L/R (SCRITTURA AL FINE)
T → TESTO
B → BINARIO

④ CERRAR ARCHIVO $\rightarrow$ FCLOSE

FCKOBJE (online 1) → SAVETS
 FILE * online 1
 online 1 = FOPEN ("EXAMPLE.TXT", "WT");
 FPUTS ("VIRGILIS", online 1)
 FCLOSE (online 1)

Funciones de control de archivos

```
FILE* archivo1;
archivo2 = fopen("Ejemplo2.txt", "w");
if (archivo1 == NULL) {
    printf("No se pudo abrir el archivo");
    exit(1); // Se sale con una señal de error
}
fwrite(archivo1, "%c", 1, 'A');
fclose(archivo1);
```

fread(archivo1)

```
fscanf(archivo1, "%s", frase);
fprintf(archivo2, "%s", frase);
```

ARCHIVO1.TXT

fread

ARCHIVO2.TXT

fwrite

~~FILE* archivo1;~~
" " " 2;

char letra1;

char letra2;

archivo1 = fopen("Ejemplo1.txt", "r");

if (archivo1 == NULL) {

printf("Error");

exit(1);

archivo2 = fopen("Ejemplo2.txt", "w");

if (archivo2 == NULL) {

printf("Error");

exit(1);

fscanf(archivo1, "%c %c", &letra1, &letra2);

fprintf(archivo2, "%c %c", letra1, letra2);

fclose(archivo1);

fclose(archivo2);

Abro Archivo

fopen

- Leer

fscanf / fgetc

→ escribir

- Escribir

fscanf / fgetc

→ leer

Cerrar Archivo

fclose


```

INT main() {
    char Nombre[20] = " ";
    PRINTF("DAME TU NOMBRE: \n");
    GETS(Nombre);
    FILE *archivos;
    archivos = FOPEN("Nico.txt", "W"); // ESCRIBIR AL NICO.TXT
    FPRINTF(archivos, "%s", Nombre);
    FCLOSE(archivos);
}

```

```

char NombreLeido[20] = " ";
archivos = FOPEN("Nico.txt", "R"); // LEER AL NICO.TXT
FGETS(NombreLeido, 20, archivos);
FCLOSE
}

```

```

STRUCT PERSONA {
    char Nombre[20];
    INT EDAD;
    float ALTURA
} PC[2];

```

```

PC[0].EDAD = 20; PC[0].ALTURA = 1.03F;
strcpy(PC[0].Nombre, "Nico");
// 20, 1.03, Nico \n
FILE *archivos;
archivos = FOPEN("Cach.txt", "W");
FPRINTF(archivos, "%d", "%2F", "%s\n", PC[0].EDAD, PC[0].ALTURA, PC[0].Nombre);
FCLOSE(archivos);

```

```

PC[1].EDAD = 30; PC[1].ALTURA = 2.03F;
strcpy(PC[1].Nombre, "Solange");
// 30, 2.03, Solange \n
archivos = FOPEN("Cach.txt", "A");
FPRINTF(archivos, "%d", "%2F", "%s\n", PC[1].EDAD, PC[1].ALTURA, PC[1].Nombre);
FCLOSE(archivos);

```

```

archivos = FOPEN("Cach.txt", "R");
char AUX[60] = " ";
WHILE (!FEOF(archivos)) {
    FGET(AUX, 60, archivos); // LEER (VAR EL PROPIETARIO)
    PRINTF("\n PERSONA ES: %s", AUX);
}

```

```
FILE ARCH, "A1", "B3";
```

```
ARCH = FOPEN("ARCHIVO.TXT", ...);
```

```
IF FEOF(ARCH) ...;
```

```
IF FERROR(ARCH) ...; SI HAY ERROR EN ALGUNA PARTE
```

```
F_CLOSE(ARCH);
```

ARCHIVO SECUENCIAL → LEER UN REGISTRO TAN OTRA (GENERADO COMO UN DISCO).

SI TIENEN 2 CAMPOS → A, B

```
FPRINTF("JUAN", 2380, 350, "BANK");
```

```
Fscanf("ARCH", "%s %i %i", NOMBRE, DNI, PROV);
```

```
REWIND(ARCH); // REBOBINA TODA EL PROGRAMA (CADA PERSONA) Y VA AL INICIO.
```

```
REMOVE("ARCHIVO.TXT"); // BORRA EL ARCHIVO.
```

REGISTROS DE LONGITUD FIJA:

1	R	2	2	3	H
0					
		32	64		96

LR	NR
32	1
33	2

15	R	28	2	A	3	H	W
		32	64				96

A	33	3	64
A2		15	1
		28	33

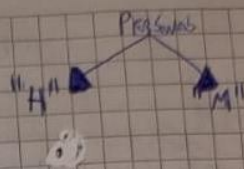
→ PARA BASE DE DATOS

```
FGETC // LEER Y GRABAR UN CARACTER A LA VEZ (AUNQUE GET NO ES LO MISMO Q PUT)
```

```
FGETS // " " " " STRING (CADENA DE CARACTERES).
```

```
FSEEK(ARCH, CONT, SEEK_SET // AL PRINCIPIO  
SEEK_CUR // " " AL ACTUAL  
SEEK_END // " " AL FINAL);
```

```
A = FTELL(ARCH) // TE DICE EN Q DIRECCION ESTAS
```



NOMBRE	FUNCION
FOpen()	ABRE UN ARCHIVO
FClose()	CERRA UN ARCHIVO
FGets()	LEE UNA CADENA DE UN ARCHIVO
Fputs()	ESCRIBE UNA CADENA EN UN ARCHIVO
Fseek()	BUSCA UN BITA ESPECIFICO DE UN ARCHIVO
Fprintf()	ESCRIBE UNA SALIDA CON FORMATO EN EL ARCHIVO
Fscanf()	LEE UNA ENTRADA CON FORMATO DESDE EL ARCHIVO
FEOF()	DEVUELVE CERO SI SE LLEGA AL FINAL DEL ARCHIVO
Ferror()	DEVUELVE CERO SI SE PRODUCE UN ERROR
rewind()	COLoca EL LOCALIZADOR DE POSICION DEL ARCHIVO AL PRINCIPIO DEL MISMO
remove()	BORRA UN ARCHIVO
unlink()	VACIA UN ARCHIVO

F: SE ABRE EL FILE PARA LECTURA; SI NO EXISTE SE DEVUELVE UN PUNTERO NULO
W: SE CREA EL FILE PARA ESCRITURA; SI YA EXISTE SE RA ELIMINADO (LO REESCRIBE)
A: SI EXISTE EL FILE SE ABRE PARA ESCRITURA (AL FINAL DEL FILE); SI NO EXISTE SE CREA
R+: SI EXISTE EL FILE, SE ABRE PARA LECTURA Y ESCRITURA (AL PRINCIPIO DEL FILE)
W+: SE CREA PARA LECTURA Y ESCRITURA (SI EXISTE LO REESCRIBE)
A+: SI EXISTE SE ABRE PARA LECTURA Y ESCRITURA (AL FINAL); SI NO EXISTE SE CREA.

t: TIPO DE TEXTO } SI NO SE ESPECIFICA; SE TOMA COMO T.
b: TIPO BINARIO }

9.4 CREA UN PROGRAMA q' LEA 2 CARACTERES DE UN ARCHIVO Y LOS COPIA EN OTRO

```
FILE *FP1, *FP2;
char C1, C2;
FP1 = fopen("archivo.txt", "r");
IF (FP1 == NULL) {
    printf("ERROR AL ABRIR EL ARCHIVO PARA LEEA");
    exit(1);
}
FP2 = fopen("copia.txt", "w");
IF (FP2 == NULL) {
    printf("ERROR AL ABRIR EL ARCHIVO COPIA.TXT");
    exit(1);
}
fscanf(FP1, "%c...%c", &C1, &C2);
fprintf(FP2, "%c...%c", C1, C2);

fclose(FP1);
fclose(FP2);
return 0;
```

while (!feof(fichero)) // ES LO MISMO A while (feof(fichero) != 0)

82) Crea un Programa q' escriba un caracter leído del teclado en un fichero

```
FILE *FP;
char caracter;
printf("Introduce caracter:");
scanf("%c", &caracter);
FP = fopen("caracter.txt", "w");
fwrite(&caracter, FP);
fclose(FP);
printf("Caracter guardado en el fichero!!!");
return 0;
```

83) Crea un Programa q' imprima un caracter leído en un fichero

```
FILE *FP;
char leído;
FP = fopen("archivo.txt", "r");
leído = fgetc(FP);
printf("%c", leído);
return 0;
```

84) Muestra la lectura y escritura de una cadena de N caracteres en un fichero

```
FILE *FP;
char cadena[100];
FP = fopen("archivo.txt", "w");
fwrite(cadena, FP);
fclose(FP);
FP = fopen("archivo.txt", "r");
fputs(leída, stdout);
fclose(FP);
printf("Cadena leída: %s", leído);
return 0;
```

85) Leer las frases q' el usuario teclea y las guarda en un fichero de texto "registro de usuario.txt". Terminar cuando la frase introducida sea "FIN" (no debe guardarse en el fichero).

```
FILE *PTFichero;
char fin[] = "FIN";
char frase[60];
PTFichero = fopen("registro de usuario.txt", "w");
printf("Escriba frases en cuando seiga 'FIN'");
```

```
do {
    puts("Escriba una frase: ");
    gets(frase);
    if(strcmp(frase, fin) == 0) break;
    printf("PTFichero: %s", frase);
```

```
while(strcmp(frase, fin) != 0);
fclose(PTFichero);
return 0;
```


86) Pida al usuario q' teclee frases y las almacene en "FRASES.TXT", acabe cuando el usuario pulse Intro sin teclear nada. Después después muestre el contenido del fichero.

```
FILE * FICHERO;
char frase[64];
int i = 0;
FICHERO = fopen("FRASES.TXT", "wt");
printf("Escriba frases, " " pulse Intro para salir");
do {
    if (i == 5) {
        printf("Escriba frase (o pulse Intro)");
        break;
    }
    printf("Escriba frase: ");
    gets(frase);
    fprintf(FICHERO, "%s", frase);
    i++;
} while (strcmp(frase, "") != 0);
printf("Lo q' escribio: ");
fclose(FICHERO);
FICHERO = fopen("FRASES.TXT", "rt");
do {
    fgets(frase, 60, FICHERO);
    puts(frase);
} while (!feof(FICHERO));
fclose(FICHERO);
printf("Hasta luego");
getchar();
fclose(FICHERO);
return 0;
```

87) Pregunte un nombre de fichero y muestre por pantalla el contenido de ese fichero (haciendo una pausa después de cada 25 líneas) para q' de tiempo a leerlo. Cuando se pulse Intro se muestren las siguientes 25 líneas y así hasta q' termine el fichero.

```
FILE * FICHERO;
char linea[100], nombre[40];
int i = 0;
do {
    printf("Nombre del fichero");
    gets(nombre);
    FICHERO = fopen(nombre, "rt");
    if (FICHERO == NULL) {
        printf("No existe el fichero");
        i++;
        if (i == 5) {
            exit(1);
        }
    }
} while (FICHERO == NULL);
```



```

WHILE (!EOF(FICHERO)) {
    FOR (T=0; T<5; T++) {
        FILAS(LINIA, 200, FICHERO);
        IF (EOF(FICHERO)) {
            PUTS(LINIA);
        }
    }
}

```

```

    GETCHAR();
}
CLOSE(FICHERO);
RETURN 0;

```

99) Crea un struct q' almacene los siguientes datos de una persona: Nombre, Edad, Ciudad de Residencia. Pasa al usuario los datos de una persona y guardarlos en "Gente.dat". Crea el fichero, búscalo y abrir para lectura y mostrar los datos q' se habían guardado.

```

struct Datos {
    char Nombre [21];
    int Edad;
    char Ciudad [21];
} Persona;
Enti main() {
    FILE * FICHERO;
    int i;
    FICHERO = FOPEN("Gente.dat", "wt");
    IF (FICHERO == NULL) {
        PRINTF("No hay archivo");
        return;
    }
}

```

```

    PUTS("Nombre: ");
    GETS(Persona.Nombre);
    FPRINTF(FICHERO, "%s", Persona.Nombre);
    PUTS("Edad: ");
    scanf("%d", &Persona.Edad);
    GETCHAR();
    FPRINTF(FICHERO, "%d", Persona.Edad);
    PUTS("Ciudad: ");
    GETS(Persona.Ciudad);
    FPUTS(Persona.Ciudad, FICHERO);
    FCLOSE(FICHERO);
}

```

Es lo mismo al usado en Nombre siendo un FPRINTF

```

FICHERO = FOPEN("Gente.dat", "rt");
IF (FICHERO == NULL) {
    PRINTF("No hay archivo");
    return;
}

```

```

while (!EOF(FICHERO)) {
    FGETS(Persona.Nombre, 60, FICHERO);
    PUTS(Persona.Nombre);
    FSCANF(FICHERO, "%d", &Persona.Edad);
    GETCHAR();
    PRINTF("%d", Persona.Edad);
    FGETS(Persona.Ciudad, 60, FICHERO);
    PUTS(Persona.Ciudad);
}

```

```

}
FCLOSE(FICHERO);
}
RETURN 0;
}

```

UNA AGENDA O' MINION. Los sig. datos: NOMBRE, DIRECCION, TEL. MOVIL, EMAIL, DÍAS, MES Y AÑO DE NACIMIENTO (DEBERÁN HABER 3 DÍGITOS N° ENTRE LOS CÍFROS). DIBUJAR TANTO LA CAPACIDAD PARA LOS FICHAS. SER DIBUJADO POR EL AUTADOR UN DIB. MUY BUENO VISUALIZAR LOS NOMBRES DE LAS FICHAS EXISTENTES, O MOSTRAR TODOS LOS DATOS DE UNA PERSONA (SE PRECISARÁ AL USUARIO CUAL ES EL NOMBRE O' SE QUIERE VISUALIZAR). AL HACER EL PROGRAMA LEERÁ TODOS LOS DATOS DE UN FICHERO "AGENDA.DAT" (SI EXISTE). AL TERMINAR, GUARDAR TODOS LOS DATOS EN EL FICHERO.

```

STRUCT AGENDA {
    CHAR NOMBRE [21];
    CHAR DIRECCION [31];
    CHAR CELULAR [13];
    CHAR EMAIL [31];
    UNSIGNED SHORT INT DIA, MES, AÑO;
} PERSONA [60];
INT MAIN() {
    FILE * FICHERO;
    CHAR TEXTO AUX [21];
    INT OPCION, PERSONAS = 0;
    INT I, j;
    IF (FICHERO != NULL) {
        WHILE (! FEOF (FICHERO)) {
            FGETS (TEXTO AUX, 20, FICHERO);
            IF (FEOF (FICHERO)) {
                BREAK;
            }
            IF (STRLEN (TEXTO AUX) > 0) {
                TEXTO AUX [STRLEN (TEXTO AUX) - 1] = '\0';
                STRCPY (PERSONA [PERSONAS]. NOMBRE, TEXTO AUX);
                PUTS (PERSONA [PERSONAS]. NOMBRE);

                FGETS (TEXTO AUX, 20, FICHERO);
                IF (FEOF (FICHERO)) BREAK;
                IF (STRLEN (TEXTO AUX) > 0) {
                    TEXTO AUX [STRLEN (TEXTO AUX) - 1] = '\0';
                    STRCPY (PERSONA [PERSONAS]. DIRECCION, TEXTO AUX);
                    PUTS (PERSONA [PERSONAS]. DIRECCION);

                    FGETS (TEXTO AUX, 20, FICHERO);
                    IF (FEOF (FICHERO))
                        BREAK;
                }
                IF (STRLEN (TEXTO AUX) > 0) {
                    TEXTO AUX [STRLEN (TEXTO AUX) - 1] = '\0';
                    STRCPY (PERSONA [PERSONAS]. CELULAR, TEXTO AUX);
                    PUTS (PERSONA [PERSONAS]. CELULAR);

                    FGETS (TEXTO AUX, 20, FICHERO);
                    IF (FEOF (FICHERO)) BREAK;
                }
                IF (STRLEN (TEXTO AUX) > 0) {
                    TEXTO AUX [STRLEN (TEXTO AUX) - 1] = '\0';
                    STRCPY (PERSONA [PERSONAS]. EMAIL, TEXTO AUX);
                    PUTS (PERSONA [PERSONAS]. EMAIL);
                }
            }
            PERSONAS++;
        }
    }
}
    
```

Notas:

[illegible]

93) Crea un programa q' lea un caracter y lo muestre por el fichero "ARCHIVO.TXT".

```
// PUNTERO A ARCHIVO
FILE *FP;
// DEFIENDO EL CARACTER PARA LEER
char LEIDO;
// ABRO EL ARCHIVO PARA LEER
FP = fopen("ARCHIVO.TXT", "r");
// LEO UN CARACTER
LEIDO = fgetc(FP);
// CIERRO EL ARCHIVO
fclose(FP);
printf("EL CARACT LEIDO ES: %c", LEIDO);
return 0;
```

94) UN CONTROL RECURSIVO CON LECTURA Y ESCRITURA

```
void ControlRecurSivo(int N1, int N2) {
```

```
if (N1 < N2) {
```

```
FILE * SALIDA;
SALIDA = fopen("SALIDA.TXT", "a");
```

```
fprintf(SALIDA, "%d\n", N1);
```

```
fclose(SALIDA);
```

```
N1++;
```

```
ControlRecurSivo(N1, N2);
```

```
}
```

```
}
```

```
int main() {
```

```
printf("LEA LOS 2 ARGUMENTOS DEL ARCHIVO\n");
```

```
FILE * ARGUMENTOS;
```

```
ARGUMENTOS = fopen("ARGUMENTOS.TXT", "r");
```

```
int NUMERO1, NUMERO2;
```

```
fscanf(ARGUMENTOS, "%d %d", &NUMERO1, &NUMERO2); // LEER LOS 2 ARGUMENTOS DEL ARCHIVO
```

```
printf("LOS NUMEROS LEIDOS DEL ARCHIVO FUERON: %d y %d\n", NUMERO1, NUMERO2); // MUESTRA LOS 2 ARGUMENTOS
```

```
fclose(ARGUMENTOS); // CIERRO LA LECTURA
```

```
FILE * SALIDA;
```

```
SALIDA = fopen("SALIDA.TXT", "w");
```

```
fclose(SALIDA);
```

```
ControlRecurSivo(NUMERO1, NUMERO2);
```

```
return 0;
```

```
ERR UN ARCHIVO CON CHAR => (NUMEROS)
```

```
char NUMERO1[3] = "111";
```

```
" 2[3] = "111";
```

```
" AUX[50] = "111";
```

```
gets(AUX[50], 50, ARGUMENTOS);
```

```
scanf("%d", &NUMERO1);
```

```
return 0;
```

```
printf("LOS NUMEROS LEIDOS SON: %d y %d", NUMERO1, NUMERO2);
```


92) MATRIZ CON ARCHIVO

```

int main() {
    FILE * FICHERO;
    int I, J; MATRIZ [10][10];

    // INICIA MATRIZ EN 0
    for (I = 1; I < 5; I++)
        for (J = 1; J < 5; J++)
            MATRIZ [I][J] = 0;

    FICHERO = fopen("origen.txt", "r");
    if (FICHERO == NULL) {
        printf("No se abre FICHERO\n");
        system("PAUSE");
        exit(EXIT_FAILURE);
    }

    I = 0;
    while (1) {
        if (feof(FICHERO))
            break;
        fscanf(FICHERO, "%d %d %d %d %d", &MATRIZ [I][0], ..., &MATRIZ [I][4]);
        I++;
    }

    fclose(FICHERO);
    printf("CONTENIDO DEL FICHERO:");
    for (I = 1; I < 5; I++)
        for (J = 1; J < 5; J++)
            printf("%d ", MATRIZ [I][J]);

    return EXIT_SUCCESS;
}

```

93) ESTRUCTURA CON ARGUMENTO

```

struct Persona {
    int dni;
    char Nombre [20];
    int Edad;
};

Personas [2], cupias [2];

void cargarPersonas (char Lectura [30], int I); // carga Perso. Usando, RECIBIR Texto y Persona A Guardar

int main() {

```

```

    Personas [0].dni = 111; strcpy (Personas [0].Nombre, "Nico"); Personas [0].Edad = 28;
    " [1].dni = 2222; strcpy (" [1] ", "Brenda"); " [1].Edad = 19;

```

```

    FILE * G;

```

Notas:

```

G= FUNDID ("Programas.Txt", "w+");
FINDUT I=0; I=2; I++;
FINDUT I=0; "day, day, day", PROGRAMAS (I).DNI, PROGRAMAS (I).NOMBRE, PROGRAMAS (I).NOMBRE);
}
FCLASS (6);
//////////////////////////////////////////////////
INT I=0;
FILE "ARCHIVO";
ARCHIVO = FOPEN (PROGRAMAS, "rt", "r");
WHILE (! FEOF (ARCHIVO)) {
    CARGA LECTURA [30];
    FPRINTF (LECTURA, "%d\n", ARCHIVO);
    IF (! FEOF (ARCHIVO)) {
        FPRINTF ("AL ARCHIVO %d", LECTURA);
        CARGA PROGRAMAS (LECTURA, I);
        I++;
    }
}
// CARGA LA LECTURA DEL ARCHIVO
// MUESTRA LO q' CARGA DESDE EL ARCHIVO
FOR (TUT INDICE=0; INDICE < I; INDICE++) {
    FPRINTF ("Programa cargada con DNI, %d, EDAD, %d, y NOMBRE %s", COPIAS [INDICE].DNI, ...);
}
}
RETURN 0;
}

// CARGA PROGRAMA (CARGA LECTURA [30], TUT I)
INT TAMANIO = strlen (LECTURA);
CARGA DNI [9] = ""; CARGA EDAD [5] = ""; CARGA NOMBRE [25] = "";
TUT FIN EDAD;
TUT FIN DNI;

// Para Pasa a Contar la Primera Coma
FOR (TUT I=0; I < TAMANIO; I++) {
    IF (LECTURA [I] == ',') {
        FIN DNI = I;
        I = TAMANIO;
    }
}

// 2da Coma
FOR (TUT I=FIN DNI+1; I < TAMANIO; I++) {
    IF (LECTURA [I] == ',') {
        FIN EDAD = I;
        I = TAMANIO;
    }
}

// Carga DNI
FOR (TUT CARGA=0; CARGA < FIN DNI; CARGA++) {
    DNI [CARGA] = LECTURA [CARGA];
}

// Carga EDAD
FOR (TUT CARGA=FIN DNI+1; CARGA < FIN EDAD; CARGA++) {
    EDAD [CARGA-FIN DNI-1] = LECTURA [CARGA];
}

// Carga NOMBRE
FOR (TUT CARGA=FIN EDAD+1; CARGA < TAMANIO; CARGA++) {
    NOMBRE [CARGA-FIN EDAD-1] = LECTURA [CARGA];
}
}

```


94) TDA con Archivos

4H

```

STRUCT EMPLEADO ESTRUCTURA;
TYPEDF STRUCT EMPLEADO ESTRUCTURA * EMPLEADO;

EMPLEADO CREAR EMPLEADO (CHAR NOMBRE[20], CHAR APELLIDO[20], INT CODIGO FUEVITO);
VOID MOSTRAR EMPLEADO (EMPLEADO E);

INT GET CODIGO FUEVITO (EMPLEADO E);

```

// C

#include "H"

```

STRUCT EMPLEADO ESTRUCTURA {
    CHAR NOMBRE[20]; CHAR APELLIDO[20]; INT CODIGO FUEVITO;
};

```

```

EMPLEADO CREAR EMPLEADO (CHAR NOMBRE[20], CHAR APELLIDO[20], INT CODIGO FUEVITO) {
    EMPLEADO E = NULL; C (sizeof (STRUCT EMPLEADO ESTRUCTURA));
    strcpy (E->NOMBRE, NOMBRE);
    strcpy (E->APELLIDO, APELLIDO);
    E->CODIGO FUEVITO = CODIGO FUEVITO;
    RETURN E;
}

```

```

VOID MOSTRAR EMPLEADO (EMPLEADO E) {

```

```

    printf (" EMPLEADO : %s, %s, FUEVITO N° : %d", E->APELLIDO, E->NOMBRE, E->CODIGO FUEVITO);
}

```

```

INT GET CODIGO FUEVITO (EMPLEADO E) {
    RETURN E->CODIGO FUEVITO;
}

```

// F.H

```

STRUCT FUEVITO ESTRUCTURA;
TYPEDF " " " " * FUEVITO;

```

```

FUEVITO CREAR FUEVITO (CHAR NOMBRE[20], INT CANT INVITADOS, INT CODIGO);
VOID MOSTRAR FUEVITO (FUEVITO E);

```

```

CHAR * GET NOMBRE (FUEVITO E);
INT GET CANT INVITADOS (FUEVITO E);
INT GET CODIGO (FUEVITO E);
INT GET EMPLEADOS CARGADOS (FUEVITO E);

```

```

VOID SET NOMBRE (" ", CHAR NOMBRE[20]);
VOID SET CANT INVITADOS (" ", INT CANT INVITADOS);
VOID SET CODIGO (FUEVITO E, INT CODIGO);

```

[illegible]


```

// Funciones
void crearVectorEventos() {
    FILE * f;
    evento = fopen("eventos.txt", "w");
    // Funcion para agregar vector eventos.
    int i = 0;
    while (!feof(evento)) {
        char aux[lectura[100]] = "";
        fgets(aux, lectura[100], evento);
        evento = crearEventoDesdeCaras(aux, lectura);
        vectorEventos[i] = e;
        i++;
    }
    fclose(evento);
}

evento crearEventoDesdeCaras(char aux[100]) {

```

```

    int ubicacionPrimerDeUnidad = 0;
    // " Segundo " = 1;
    char aux[100];
    // " Cant " = " "
    // " Cod " = " "
    for (int j = 0; j < 100; j++) {
        if (aux[j] == ',') {
            ubicacionPrimerDeUnidad = j;
            i = 100;
        }
    }
}

```

// Ver Texto en PC.

95) APOGEO DE ARCHIVO

```

printf("APOGEO ARCHIVO");
FILE * Archivo1;
FILE * " 2";
FILE * " 3";

Archivo1 = fopen("N1.txt", "r");
" 2 = " ("N2", "r");
" 3 = " ("N3", "w");
fclose(Archivo3);
" 3 = fopen("Numero3.txt", "a");

```

// Variables Logicas
 int usar1 = 1, usar2 = 1;

Notas:

```

INT AUX1=0;
CHAR AUX1_CHAR[4]=" ";
INT AUX2=0;
CHAR AUX2_CHAR[4]=" ";

WHILE (1) {
    IF (USUARIO==1) {
        FGETS (AUX1_CHAR, 4, ARCHIVO1);
        AUX1 = atoi (AUX1_CHAR);
    }
    IF (USUARIO==2) {
        FGETS (AUX2_CHAR, 4, ARCHIVO2);
        AUX2 = atoi (AUX2_CHAR);
    }
    IF (AUX1==AUX2) {
        FPRINTF (ARCHIVO3, "%d", AUX1);
        LRF2=0;
        LRF1=1;
    }
    // RECURSIVA caso contrario. (i= No. / <5)
    IF (FEOF (ARCHIVO2)) {
        FPRINTF (ARCHIVO3, "%d", AUX2);
        WHILE (!FEOF (ARCHIVO1)) {
            FGETS (AUX1_CHAR, 4, ARCHIVO1);
            AUX1 = atoi (AUX1_CHAR);
            FPRINTF (ARCHIVO3, "%d", AUX1);
        }
        Break;
    }
    // Continúa PROC. AL REVRS TF CON ARCHIVO1 Y WHILE CON ARCHIVO2.
}
Break;
}
RETURN 0;
}

```

96) CODIGO POSTAL CON ARCHIVOS

// ESTRUCTURA LOCALIDAD ALMACENADA EN EL ARCHIVO LOCALIDADES.TXT

```

STRUCT LOCALIDAD {
    CHAR COD[2];
    " Nombre[20];
    " CodPostal[40];
}

```

// E. PERSONA EN ARCHIVO PERSONA.TXT Y TIENE E. ASOCIADA.

```

STRUCT PERSONA {
    CHAR DNI[8];
    " Nombre[30];
    " DIRECCION[70];
    STRUCT LOCALIDAD Loc; // LA ANTERIOR ESTRUCTURA.
}

```


1) Pido los datos por pantalla

// APP LOCALIDADES PARA LISTAR

WAVE (1" FOF (LOCALIZADOS)) & 6 (FICHADES = 1") 5//MILITARES No 3000 FICHADES → No 3000 FIM DE REGISTRO

TUT UB: CAJON PRIMORDIAL LIMITADA = 4;

1/ Trilha Lda Campos Verdes

STRCPV ($L = \text{CoD}$, " " " ");

11 (London), 11 11

11/10/1998

LEO LUNA IN LINEA
FLOT (1912-1913)

FGF15 (LIM1 LOC 232121) =

1/24/16 - 9:00 PM DEFINITADOR

$$E_p(t) = \rho \cdot T \leq 100 \cdot T$$

TE (UNPA) LOCALIDAD

VERTICAL PRIMER DRUM

$$T = 100^\circ$$

3

Ex. 10. AL 2 Do.

$GO(T) = \text{VERIFICAÇÃO PRIMO}$

OK 1-1-1

```

3 GUARDO LO PRIMERO AL COD
FOR (I=0; I<UBRACION PRIMER DELIMITADOR; I++) {
    L.COD[I] = UNRA LOCALIDAD [I];
}
3
INT InicioNombre = 0;
// GUARDO EL NOMBRE
FOR (I = UBRACION PRIMER DELIMITADOR + 1; I<UBRACION SEGUNDO DELIMITADOR; I++) {
    L.Nombre [InicioNombre] = UNRA LOCALIDAD [I];
    InicioNombre++;
}
3
INT InicioCodPostal = 0;
// GUARDO POR ULTIMO EL CODIGO POSTAL
FOR (I = UBRACION SEGUNDO DELIMITADOR + 1; I<I20; I++) {
    L.CodPostal [InicioCodPostal] = UNRA LOCALIDAD [I];
    InicioCodPostal++;
}
3
IF (strcmp (Cod Buscado, L.Cod) == 0) {
    Encuentrado = 1;
}
3 // SI EL CODIGO BUSCADO ES = AL UPRDO, PONGO ENCUENTRADO EN 1
3
IF (Encuentrado == 1) {
    P.Loc = 1;
}
3 ELSE {
    strcpy (P.Loc.Nombre, "SIN LOCALIDAD");
    strcpy (P.Loc.CodPostal, "SIN LOCALIDAD");
}
3
FCLOSE (LOCALIDADES);
PRINTF ("%s, %s, %s, %s", P.DIRE, P.Nombre, P.DIRECCION, P.Loc.Nombre, P.Loc.CodPostal);
FPRINTF (PTRICARRO, "%s, %s, %s, %s", P.DIRE, P.Nombre, P.DIRECCION, P.Loc.Nombre, P.Loc.CodPostal);
RETURN 0;
3

```